Indirect Denial-of-Service via Health-Check Manipulation in Docker Swarm

A Restart-Storm Availability Attack on Containerized Microservices
— and its Detection and Mitigation

Course	Information Security (CEP-2026)
Department	Software Engineering, NED University of Engineering & Technology
Instructor	Miss Sheerina
Group	B14

Group Members

Name	Reg. No.
Muhammad Abdullah Khan	SE-23061
Furqanullah	SE-23063
Masroor Hassan	SE-23064
Haris Masood	SE-23088
Muhammad Huzaifa Hassan	SE-23089
Muhammad Rafay	SE-23100

Single-node Docker Swarm testbed. All metrics measured on the authors' own host.

Contents

Abstract 2

1 Introduction and Motivation 2

2 Problem Description — The Indirect DoS and the Restart Storm 2

3 Relevance to Containerized Environments 3

4 Threat Model 3

5 Testbed Design 3

6 Attack Demonstration — Method and Evidence 5

7 Detection 8

8 Mitigation 9

9 Results 10

10 Conclusion and Future Work 12

11 References 12

A Reproducing the Results 13

B Additional Dashboard Views 14

Abstract

In Docker Swarm, a single `HEALTHCHECK` drives *two* independent decisions at once: whether a task receives traffic, and whether a task is restarted. When that health-check depends on an external resource such as a database, an attacker who merely *slows the dependency* — without ever touching the victim container — makes the health-check time out. Swarm's `restart_policy`, behaving exactly as designed, then kills and recreates the task again and again: a **restart storm** that takes the service offline even though the application code is perfectly functional.

We reproduced this attack in a controlled single-node Swarm testbed, built a real-time detector that identifies the storm and names its cause, and applied a mitigation that separates process liveness from dependency health. Measured before and after, the same attack drops `/work` availability to **21%** with **2** forced restarts on the vulnerable stack, but leaves availability at **100%** with **0** restarts on the mitigated stack. The detector raises its first alert **17.7 s** after the attack begins and correctly stays silent when no storm occurs.

1 Introduction and Motivation

Container orchestrators health-check every service and act on the result. In Docker Swarm the same probe is overloaded: a failing probe simultaneously removes the task from the load-balancer rotation *and* feeds the restart decision. This coupling is convenient, widely taught, and ships by default — the canonical tutorial health-check pings the database. It is also the root of the vulnerability studied here.

The insight of this project is that **availability can be destroyed without attacking the victim at all.** The application process can be healthy, answering real requests correctly, and still be killed repeatedly by its own orchestrator because a *dependency* it probes has been slowed. The attacker's capability required is minimal: the ability to add latency to one network path.

2 Problem Description — The Indirect DoS and the Restart Storm

The failure chain has five stages and never touches the victim container:

1. The attacker slows a dependency (here, the database path) so responses take longer than the health-check timeout.
2. The service's `/health` probe performs a database round-trip and times out.
3. After the configured retries, Docker marks the container **unhealthy**.
4. Swarm treats the unhealthy task as failed and, under a `restart_policy` with no `max_attempts`, terminates and recreates it.
5. The replacement task boots, fails the same dependency-coupled probe, and is killed again — a continuous terminate-and-restart cycle (the *restart storm*). The service is unavailable for the duration.

The defining fingerprint — the **signature signal** — is a *divergence*: the application keeps succeeding at real work while the health-check keeps failing. In our captured logs the application logs `WORK ok` right up to each `HEALTH fail ... timeout expired`, proving the code is alive while Swarm

kills it.

3 Relevance to Containerized Environments

This is specifically an *orchestration* problem, not a generic web-app bug:

- **Orchestrator coupling.** Swarm's single health-check drives both routing and restart. One failed probe triggers two corrective actions at once; there is no separation of concerns.
- **`restart_policy` amplification.** Auto-restart converts a transient dependency blip into a sustained outage. The orchestrator actively degrades availability *while trying to restore it* — a self-inflicted DoS.
- **Microservice blast radius.** Every service whose health-check touches a degraded dependency fails together; one slow database can collapse a whole request path in seconds.
- **Default-config exposure.** The vulnerable pattern (health-check pings the database) is the documented "best practice" in tutorials and Helm charts, so the risky configuration is shipped by default.

In a monolith where health and routing are decoupled, this class of risk is absent. The orchestrator is what turns a *dependency failure* into an *availability failure*.

4 Threat Model

- **Attacker goal:** deny availability of a target service.
- **Attacker capability:** inject latency into the network path between the service and one of its dependencies (modelled here with a latency injector). No code execution, no credentials, and no access to the victim container are required.
- **What the attacker never does:** touch, exploit, or even reach the victim web container. Only the dependency path is slowed.
- **Assumptions:** the service runs a dependency-coupled health-check (the default pattern) and a `restart_policy` without a bounded `max_attempts`.
- **Out of scope:** classic volumetric/network DDoS, host compromise, multi-node cluster effects. The single-node testbed reproduces the same orchestrator behaviour without them.

5 Testbed Design

A single-host Docker Swarm (`docker swarm init`, one Ready node) runs three services on one overlay network: `web` (the Flask victim service), `db` (`postgres:16`, the dependency), and `toxiproxy` (the latency injector between `web` and `db`).

The web app exposes deliberately separated endpoints: `/work` (real work, never touches the database), `/health` (the *vulnerable* probe: a database round-trip through Toxiproxy), `/healthz` (the *mitigated* local-only liveness probe), `/ready` (database-dependent readiness, guarded by a circuit breaker), and `/metrics` (Prometheus metrics). The application reaches the database *only* through Toxiproxy, so the attack can slow the database path without touching `web`. Two stack files differ only in the two settings that matter: `vulnerable.yml` (health-check probes `/health`; `restart_policy` has no `max_attempts`) and `mitigated.yml` (health-check probes `/healthz`; `restart_policy` adds `max_attempts: 3` and a `window`).

```
PS C:\MyWorks\IS Docker Swarm Project\iddos-swarm> ./demo/reset.ps1
[reset] removing any existing 'app' stack ...
[reset] waiting for teardown ...
[reset] deploying: vuln (+dashboard)
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network app_iddosnet
Creating config app_grafana_dashboard
Creating config app_prometheus_config
Creating config app_grafana_datasource
Creating config app_grafana_dashprovider
Creating config app_toxiproxy_config
Creating service app_db
Creating service app_grafana
Creating service app_prometheus
Creating service app_toxiproxy
Creating service app_web
[reset] waiting for web to become healthy ...
[reset] web is healthy: Up 6 seconds (healthy)
[reset] done. Endpoints:
          app   http://localhost:8000/work  (and /health /healthz /ready /metrics)
          toxi  http://localhost:8474
          Grafana http://localhost:3000   Prometheus http://localhost:9090
PS C:\MyWorks\IS Docker Swarm Project\iddos-swarm>
```

Figure 1: Clean start: `reset.ps1` brings up the vulnerable stack plus the Prometheus/Grafana dashboard on a single Swarm node; the web service reaches *healthy* and all endpoints are listed.

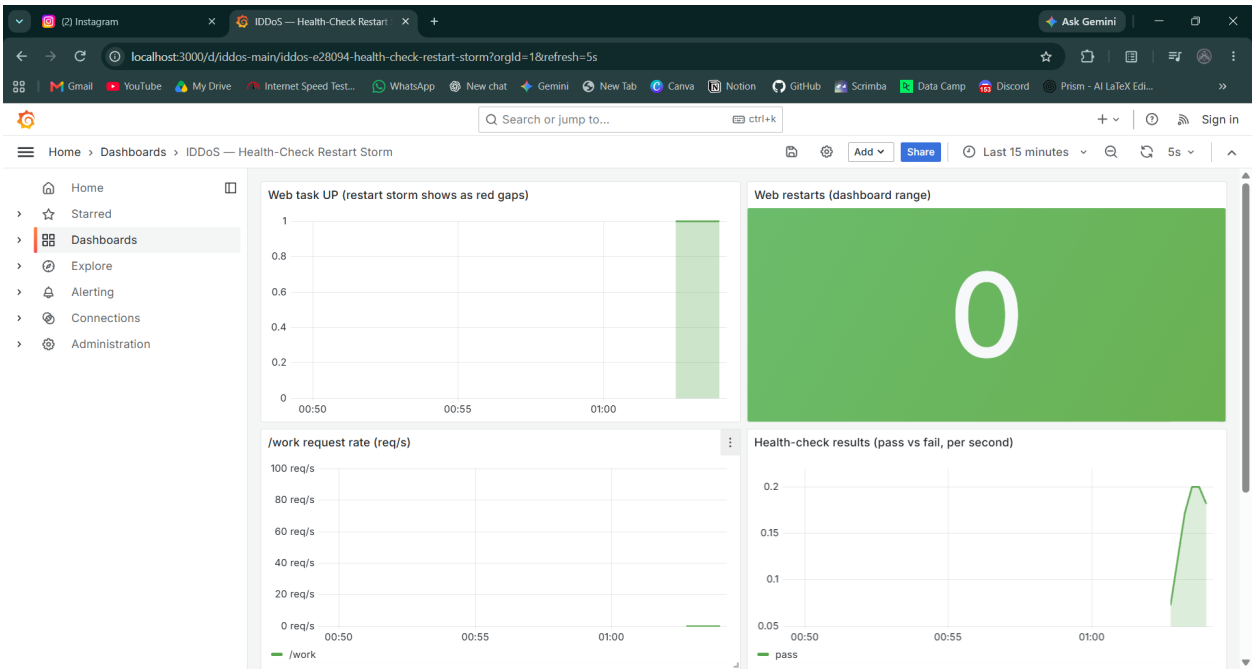


Figure 2: Baseline (no attack) on the live Grafana dashboard: the web task is continuously UP, the restart count is 0, and the request/health panels are steady.

6 Attack Demonstration — Method and Evidence

Method. The attack is a single action: add a latency *toxic* (5000 ms) to the Toxiproxy route via its control API, so database replies exceed the health-check timeout. A load generator samples `/work` throughout, and `docker stack ps` plus the service logs are captured.

Evidence. During the attack, Swarm repeatedly kills the healthy task. The task history states the cause explicitly — `"non-zero exit (137): dockerexec: unhealthy container"` (Figure 3).

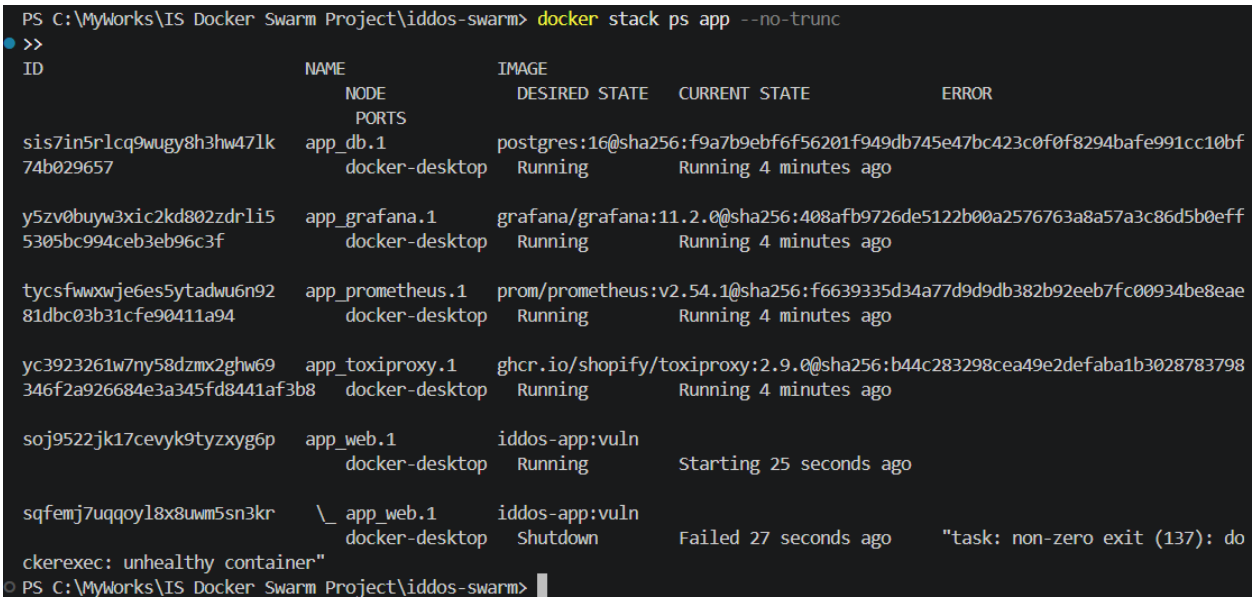


Figure 3: The restart storm in `docker stack ps app`: a replacement `app_web` task is *Starting* while the previous one is *Shutdown / Failed* with `"non-zero exit (137): dockerexec: unhealthy container"`. Swarm is killing a process that is in fact alive.

The **signature signal** is visible in the application log: real work succeeds (`WORK ok ... 200`) while the health-check fails on a database timeout (`HEALTH fail ... 503 ... timeout expired`), interleaved on the same running task (Figure 4).

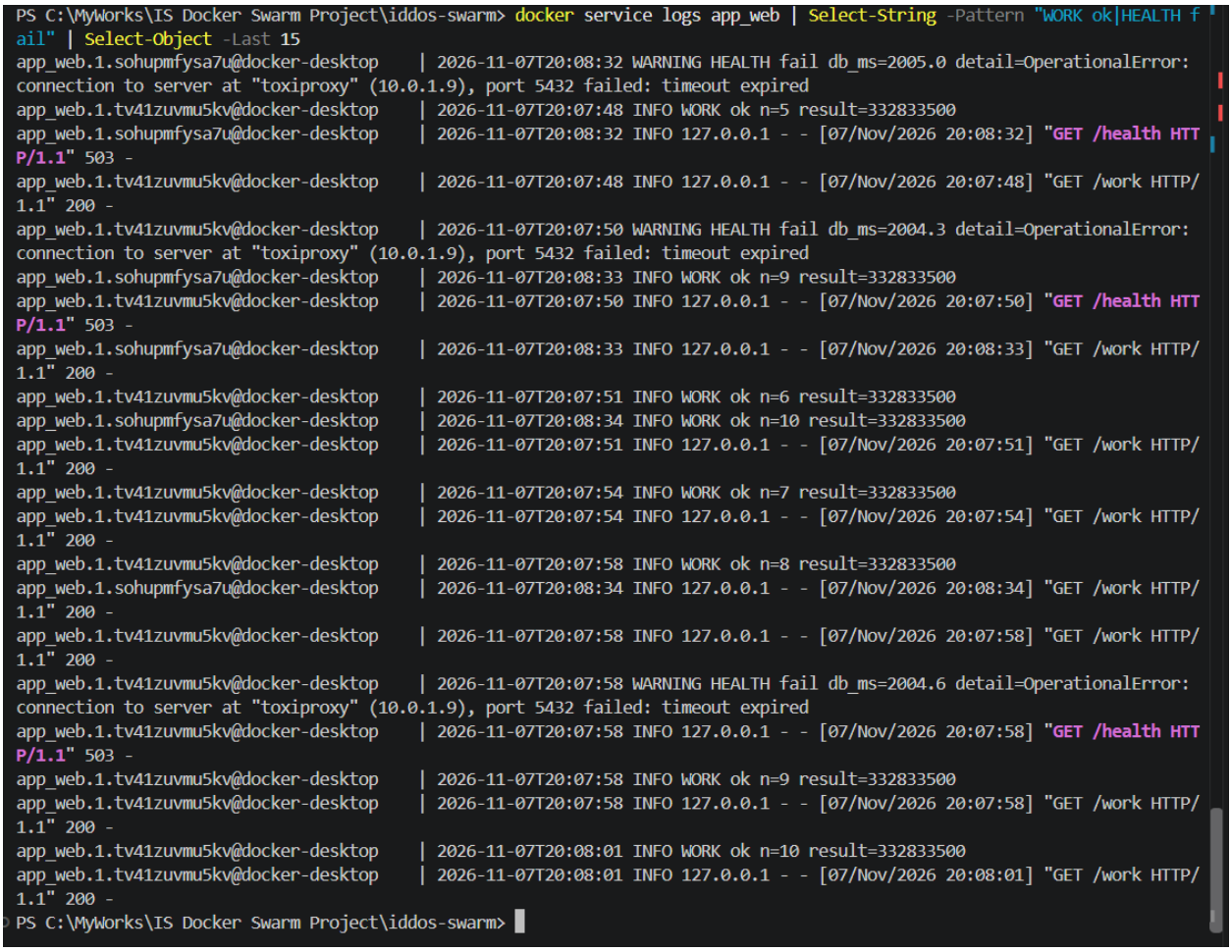


Figure 4: Signature signal in the web service log: `/work` returns 200 while `/health` returns 503 with a Toxiproxy timeout. The application code is healthy; only the dependency-coupled probe fails.

On the live dashboard the "web UP" panel shows a red gap for the period the task is down, and the health-check *fail* series rises (Figure 5); once Swarm has replaced the task, the restart counter registers the event (Figure 6).

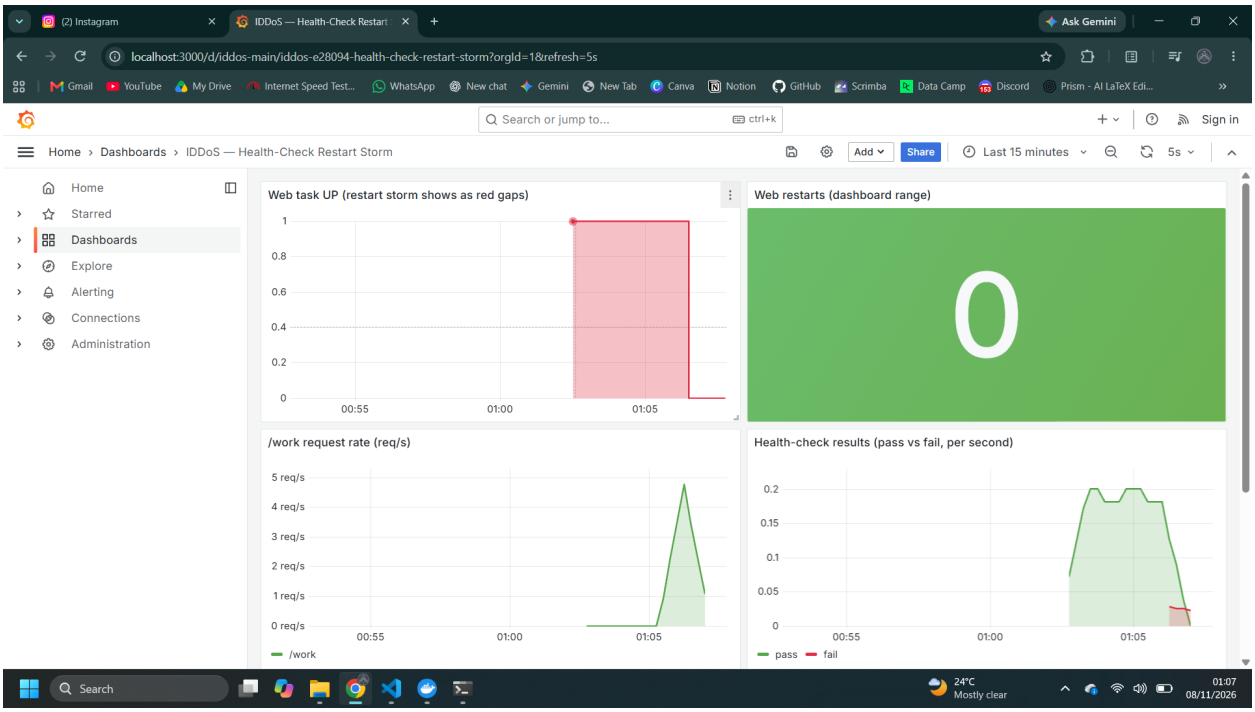


Figure 5: Grafana during the attack: the "web task UP" panel shows a red down-period (the restart gap), and the health-check panel begins recording failures.

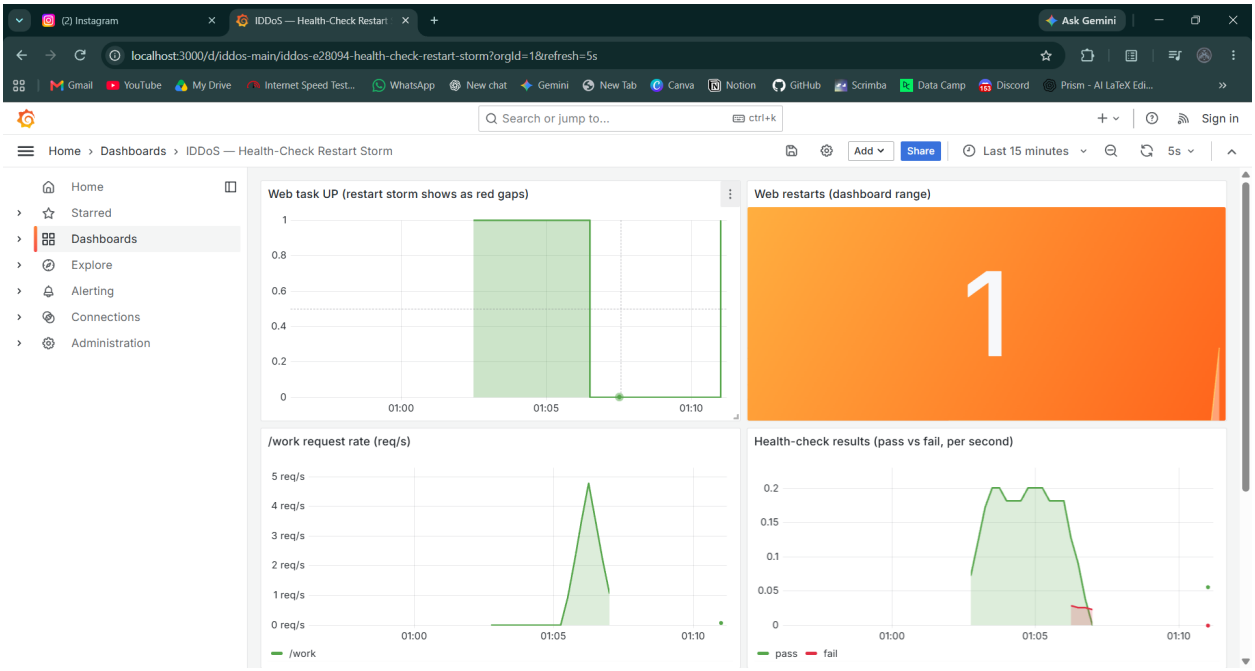


Figure 6: The restart is detected without cAdvisor: `changes(process_start_time_seconds)` increments the "Web restarts" stat as Swarm recreates the task.

When the toxic is removed, the service recovers on its own (Figure 7). The victim container was never attacked — only the database path was slowed.

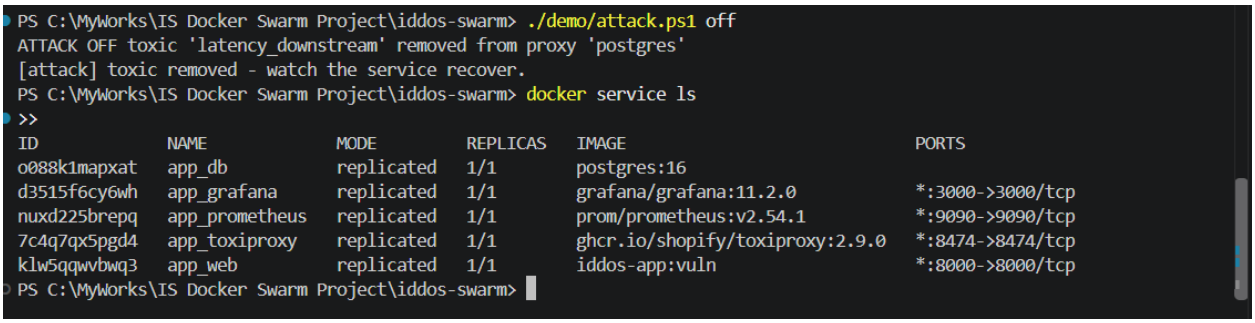


Figure 7: Recovery: after `attack.ps1 off`, `docker service ls` shows every service back at `1/1`.

7 Detection

`detector/monitor.py` (Docker SDK for Python) watches three signals in real time and writes alerts to `results/alerts.log`:

1. **Health events.** Streams container `health_status` events and counts repeated `unhealthy` transitions in a sliding window (health flapping).
2. **Restart frequency.** Polls the service tasks and counts restarts in a sliding window, alerting past a configurable threshold.
3. **Signature signal.** When health fails, it probes `/work` *inside* the container (via `docker exec`, bypassing the de-routed ingress mesh). A 200 proves the application is alive, so the detector reports an **indirect DoS** caused by dependency latency — not a crash.

The detector raised its first alert **17.7 s** after the attack began and, on the mitigated stack, correctly produced **no alerts** — there was no storm to detect. Thresholds live in a config block so the team can tune them.

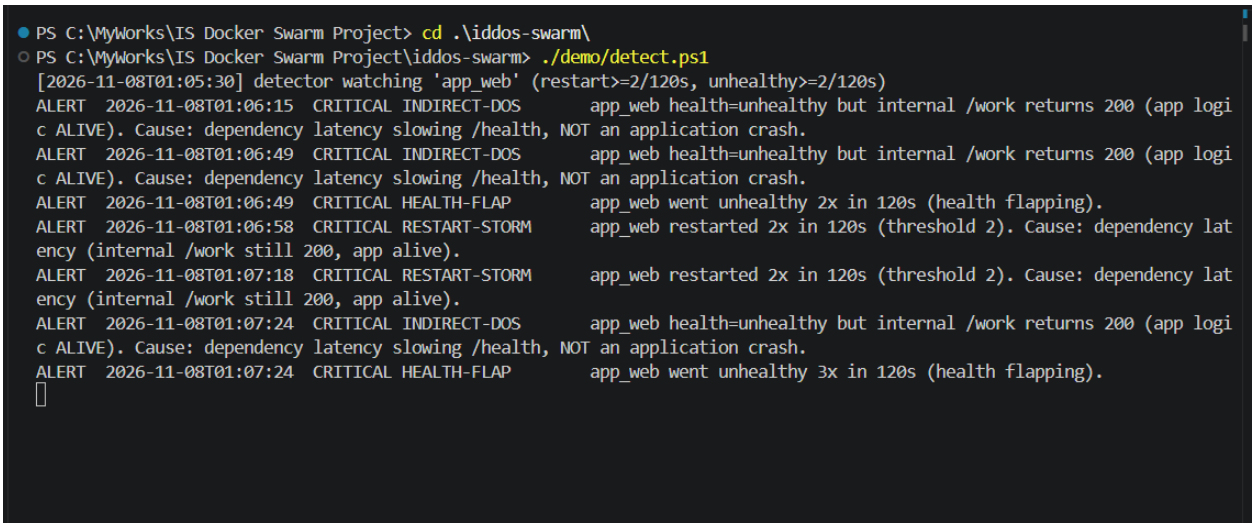


Figure 8: Live detector output: `INDIRECT-DOS`, `HEALTH-FLAP` and `RESTART-STORM` alerts, each naming the cause as dependency latency rather than an application crash.

8 Mitigation

Core fix (required).

- **Separate liveness from readiness.** The container `HEALTHCHECK` now probes `/healthz`, a local-only liveness check that does *not* touch the database. A slow dependency can no longer fail the health-check, so Swarm no longer kills a healthy container. `/ready` keeps the database check for *routing* decisions only.
- **Tune the restart policy.** `restart_policy` gains `max_attempts: 3` and a `window: 60s`, so even a genuine failure cannot become an unbounded storm.

Defence-in-depth (optional, implemented).

- **Circuit breaker** around the database call: after repeated slow calls the breaker opens and `/ready` fails fast (≈10 ms) instead of hanging for the full timeout on every request.
- **Reverse proxy (HAProxy)** that health-checks `/ready` and de-routes a slow instance *without killing it*. Captured proof under latency: `/healthz` → 200, `/ready` → 503, `/work` → 200, and the container is never killed.

Switching to the mitigated stack is a single command (Figure 9); under the *same* attack, `docker stack ps` shows no killed tasks at all (Figure 10).

```
PS C:\MyWorks\IS Docker Swarm Project\iddos-swarm> ./demo/fix.ps1
[fix] switching to the mitigated stack ...
[reset] removing any existing 'app' stack ...
[reset] waiting for teardown ...
[reset] deploying: mitigated (+dashboard)
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network app_iddosnet
Creating config app_grafana_datasource
Creating config app_grafana_dashprovider
Creating config app_grafana_dashboard
Creating config app_toxiproxy_config
Creating config app_prometheus_config
Creating service app_db
Creating service app_grafana
Creating service app_prometheus
Creating service app_toxiproxy
Creating service app_web
[reset] waiting for web to become healthy ...
[reset] web is healthy: Up 7 seconds (healthy)
[reset] done. Endpoints:
          app   http://localhost:8000/work  (and /health /healthz /ready /metrics)
          toxi  http://localhost:8474
          Grafana http://localhost:3000   Prometheus http://localhost:9090
PS C:\MyWorks\IS Docker Swarm Project\iddos-swarm>
```

Figure 9: `fix.ps1` redeploys the mitigated stack (liveness-only health-check + bounded restart policy); the web service comes up healthy.



Figure 10: The same attack on the mitigated stack (`iddos-app:fixed`): every task is *Running* with *no* `Shutdown/Failed` rows — the restart storm is gone.

9 Results

All three scenarios used the same attack (5000 ms latency), a 90 s attack window, and `/work` sampled at 5 requests/second.

Table 1: Before/after metrics across the three scenarios.

Scenario	Stack	Attack	Avail.	Restarts	MTTR (s)	TTD (s)
Baseline	vulnerable	no	100.0%	0	—	—
Vulnerable + attack	vulnerable	yes	21.0%	2	13.0	17.7
Mitigated + attack	mitigated	yes	100.0%	0	0.2	—

Metrics. **Availability** = share of `/work` requests returning 200; **Restarts** = web task restarts in the attack window; **MTTR** = seconds from the toxic being removed to the web service becoming healthy again; **TTD** (time-to-detection) = seconds from attack start to the detector's first alert ("—" = no alert, i.e. no storm to detect).

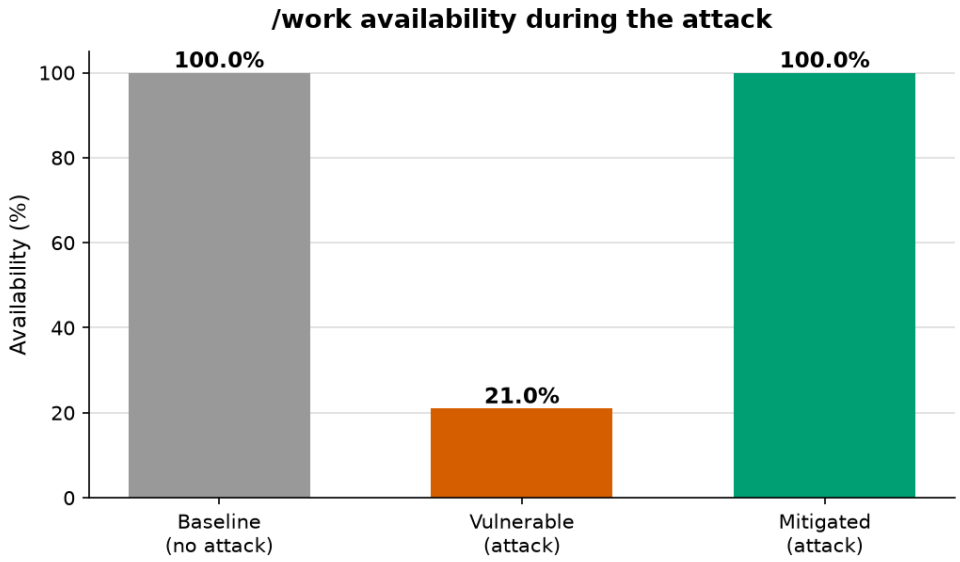


Figure 11: `/work` availability during the attack: 100% baseline, 21% on the vulnerable stack, 100% on the mitigated stack.

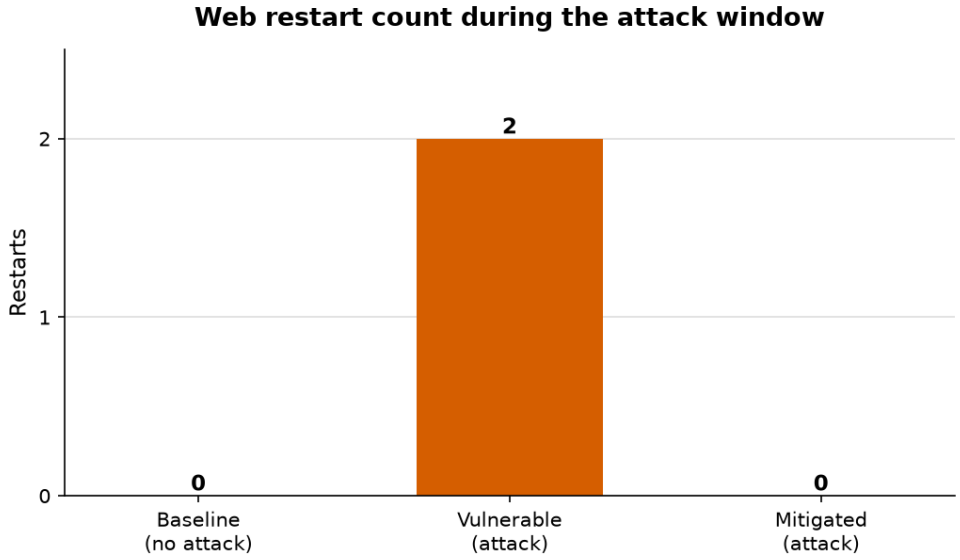


Figure 12: Web restart count during the attack window: 2 on the vulnerable stack, 0 on the mitigated stack.

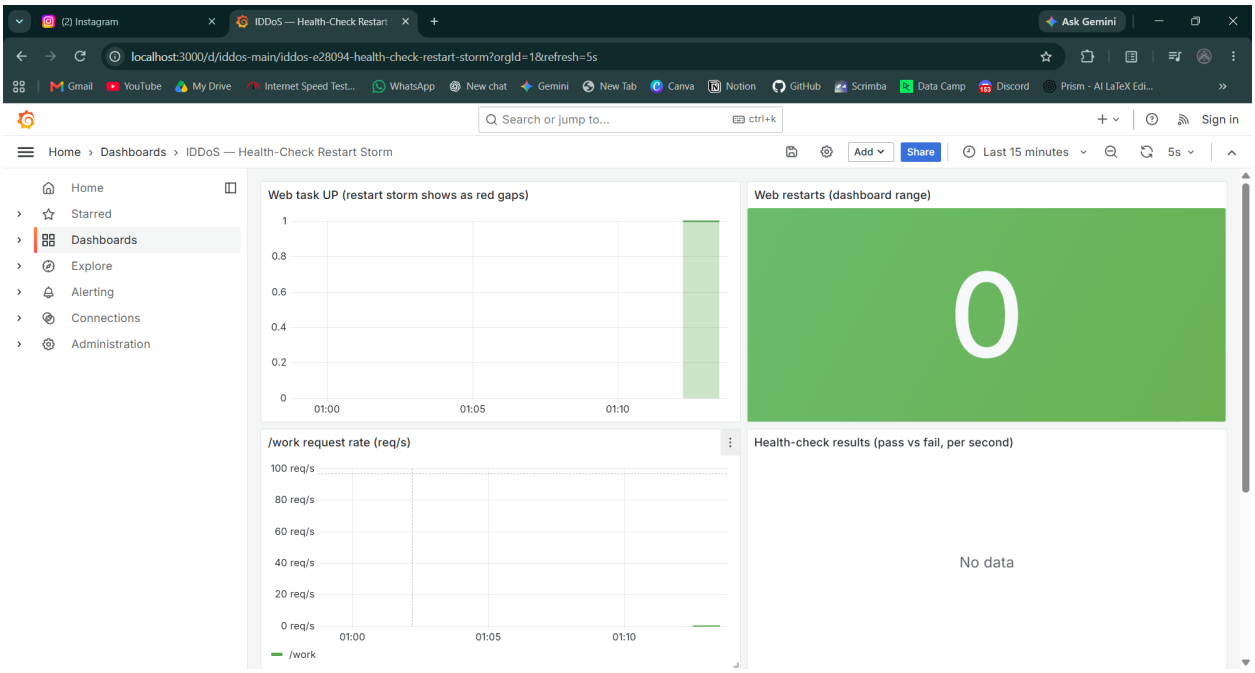


Figure 13: The mitigated stack under the same attack on the live dashboard: the web task stays UP and the restart count holds at 0.

Reading the results. On the vulnerable stack the attack is devastating: availability falls to roughly one-fifth and the orchestrator forces repeated restarts. On the mitigated stack the *same* attack is a non-event: availability holds at 100%, no restarts occur, and recovery is immediate. This meets every acceptance target set for the project (availability $\geq 95\%$, restarts low/zero, short MTTR, detection $\leq 30\,\text{s}$).

10 Conclusion and Future Work

An orchestrator feature intended to improve availability — automatic restart on a failed health-check — becomes an availability *weapon* when the health-check is coupled to an external dependency. The attack needs no exploit on the victim; it only slows a dependency. The fix is architectural and inexpensive: **separate process liveness from dependency health** and **bound the restart policy**. Our measurements show the mitigation neutralises the attack completely in this testbed.

Future work. Extend to a multi-node cluster and multiple dependent services to observe cascade/blast-radius effects; compare Swarm's behaviour with Kubernetes liveness/readiness/startup probes; add automated alerting rules in Prometheus; and study adaptive attackers that keep latency just below the detection threshold.

11 References

Container orchestration

1. Docker Inc. (2024). *Configure healthchecks in Docker Swarm.* Docker Documentation.
2. Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). *Borg, Omega, and Kubernetes.* ACM Queue, 14(1).

DoS / availability attacks

3. Carl, G., Kesidis, G., Brooks, R. R., & Rai, S. (2006). *Denial-of-Service Attack-Detection Techniques.* IEEE Internet Computing, 10(1).
4. Peng, T., Leckie, C., & Ramamohanarao, K. (2007). *Survey of network-based defense mechanisms countering the DoS and DDoS problems.* ACM Computing Surveys, 39(1).

Microservice & cascade failures

5. Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. O'Reilly Media.
6. Richardson, C. (2019). *Microservices Patterns.* Manning Publications. (Chapter on circuit breakers.)
7. Aceto, G., Botta, A., Marchetta, P., Persico, V., & Pescapè, A. (2018). *Cloud monitoring: A survey.* Computer Networks, 57(9).

A Reproducing the Results

From the `iddos-swarm/` directory (see `README.md` and `demo/runbook.md` for detail):

```
# build images
docker build -t iddos-app:vuln app/
docker build -f app/Dockerfile.fixed -t iddos-app:fixed app/

# full measured comparison (writes results/metrics.csv + charts)
python load/measure.py
python load/make_charts.py

# live demo (PowerShell on Windows)
.\demo\reset.ps1          # vulnerable stack + dashboard (http://localhost:3000)
.\demo\attack.ps1         # start the attack;  .\demo\attack.ps1 off to stop
.\demo\fix.ps1            # switch to the mitigated stack
```

B Additional Dashboard Views

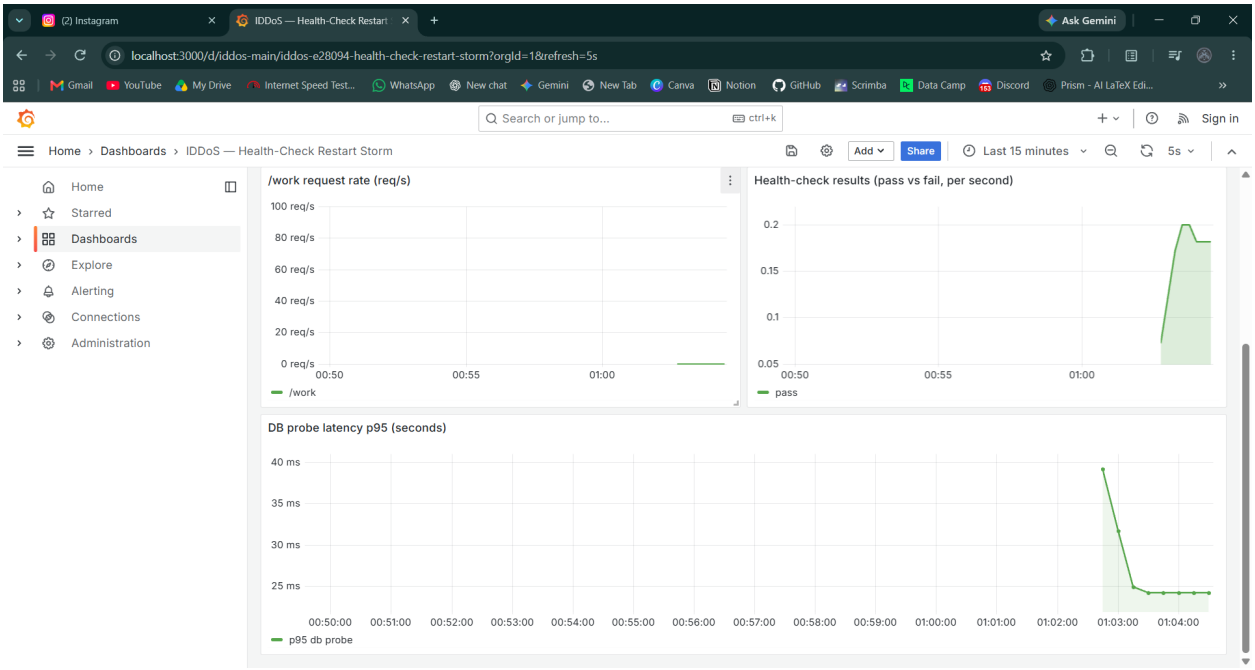


Figure 14: Baseline dashboard (lower panels): `/work` request rate, health-check pass/fail, and DB probe latency.

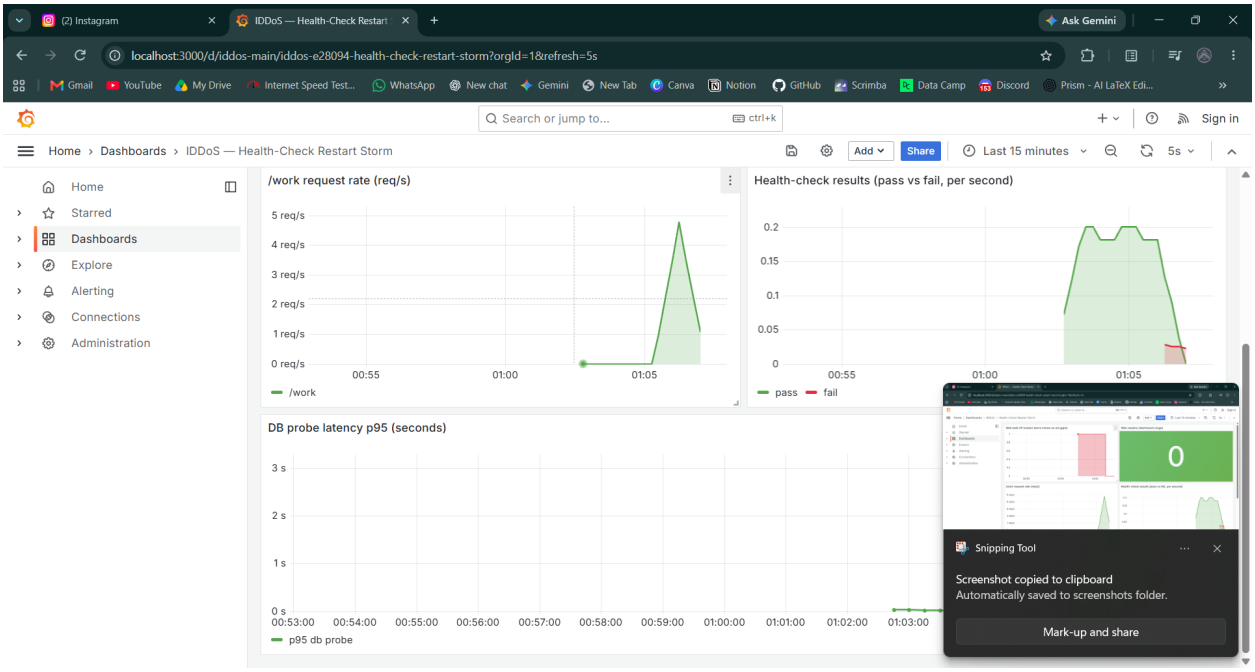


Figure 15: During the attack (lower panels): the health-check *fail* series appears and DB probe latency spikes toward the injected delay.

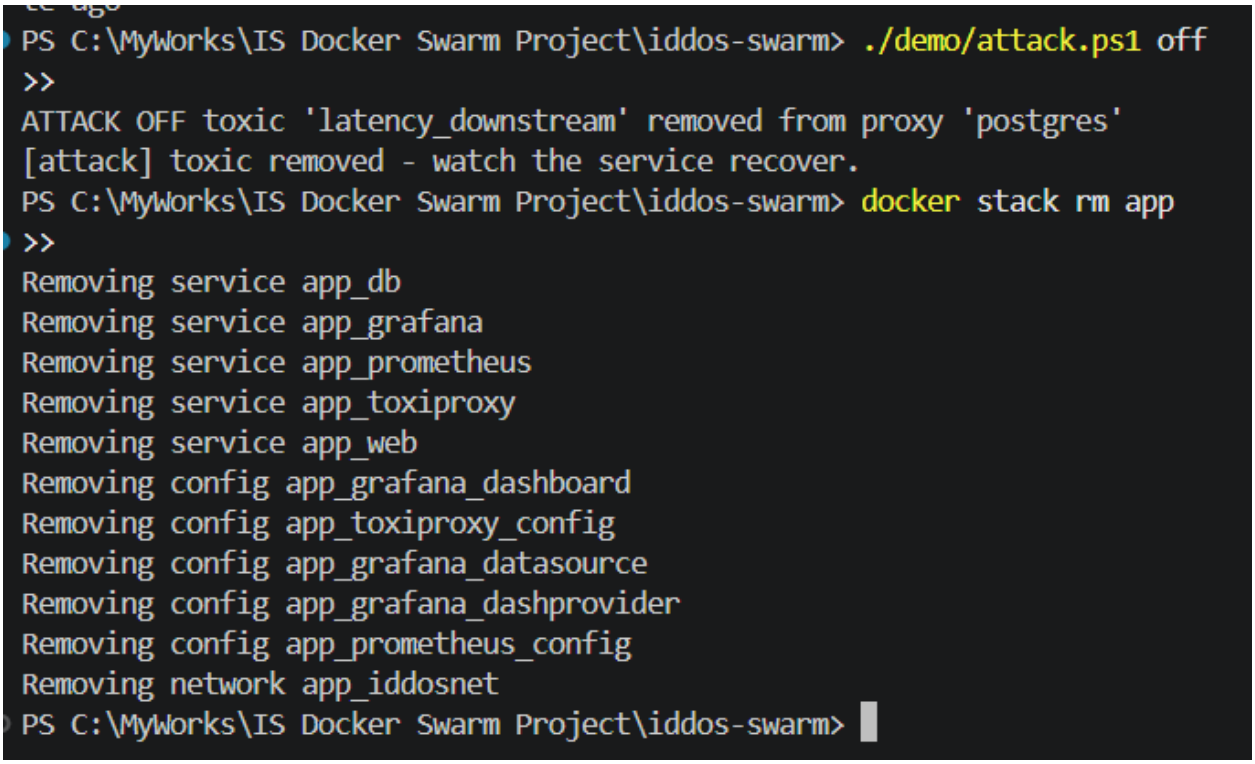


Figure 16: Teardown: `attack.ps1 off` followed by `docker stack rm app` removes all services, configs and the overlay network.